Multiplicity-Theoretic HR Matchmaking (DRME-X) — Formal Specification

0. Purpose and scope

DRME-X is a matching system between job seekers and employment opportunities that represents candidates, roles, and organizational context as **multiplicity states** over a **prime-indexed ontology**. Matching is computed via a **resonance functional** that combines (i) intersection multiplicity, (ii) critical-constraint penalties, (iii) growth alignment, and (iv) hypergraph context compatibility.

This document formalizes: - the **prime ontology** (feature universe and how primes are assigned/managed), - **evidence rules** (how signals update multiplicities), - **uncertainty representation** (confidence, partial observability), - stable **operator dynamics** (bounded updates), - the **resonance functional** used for ranking.

---

1. Core mathematical objects

1.1 Feature universe and prime index

Let F be the set of all atomic features. Each $f \in F$ is assigned a unique prime label p_f. In implementation, primes act as **stable unique IDs** (we do not multiply primes; we store sparse exponent vectors).

1.2 Multiplicity state space

A **multiplicity state** is a nonnegative vector over features:

$$x \in \mathbb{R}_{\geq 0}^{F} \quad (\text{sparse in practice})$$

Interpretation: x_f is the **multiplicity/exponent weight** of feature f for a candidate/role at time t.

We maintain: - Candidate i: $x_i(t)$ - Role j: $y_j(t)$ - Candidate growth/trajectory: $g_i(t)$ - Candidate context preference/history: $c_i(t)$ - Role context signature (team/project): $c_j(t)$

1.3 Hypergraph context

Let $H(t) = (V, E(t))$ be a hypergraph. - Nodes V: candidates, roles, teams, projects, departments, skill-communities - Hyperedges $E(t)$: multi-way relations (team composition, project staffing, workflow adjacency)

Define a context aggregation operator Agg_H producing a feature-vector signal from a node's neighborhood.

---

2. Prime ontology

2.1 Ontology layers

DRME-X uses a structured ontology with four primary layers:

1) **Hard skills / tools / domains** - e.g., Python, SQL, Kubernetes, Oncology, Accounting

2) **Soft traits / behaviors** (never purely self-declared) - e.g., collaboration, ownership, conflict navigation

3) **Values / preferences / constraints** - e.g., remote-first, mission alignment, travel tolerance, salary bands

4) **Context features** - e.g., team norms, management style, cadence, product lifecycle stage

Each feature f has metadata: - `name`, `description` - `type` $\in \{\text{hard}, \text{soft}, \text{value}, \text{context}\}$ - `parent` (taxonomy) - `synonyms` - `evidence_policy` (how multiplicity is updated) - w_f (importance weight for intersection) - u_f (penalty weight if critical and missing)

2.2 Prime assignment policy

Primes are assigned to features by an ontology service. In implementation they may be stored as integer IDs, but the policy is written as "prime allocation."

Invariants 1. **Uniqueness**: each atomic feature maps to exactly one prime ID. 2. **Stability**: once assigned, a prime ID never changes. 3. **Non-reuse**: retired features are never reassigned. 4. **Traceability**: every feature records provenance (who/when/why created).

Creation criteria (when to add a new prime) A new feature f is added if and only if: - it is **operationally distinct** (would change a hiring decision in plausible cases), and - it has **measurable evidence pathways** (Section 3), and - it cannot be represented as a synonym or alias of an existing feature.

Synonyms and aliasing - Multiple surface forms (e.g., "PyTorch", "Torch") map to the same feature f. - Close but distinct skills (e.g., "TensorFlow" vs "PyTorch") remain separate features but may be connected via a **similarity graph** (optional geometric layer).

2.3 Ontology evolution and versioning

- The ontology is versioned $\mathcal{O}_t$.
- Matching logs record the ontology version used.
- New features are appended; no destructive edits to meaning.

---

3. Evidence rules (how multiplicities are set and updated)

3.1 Evidence types

Evidence e arrives as structured events with source and reliability: - $e \in \mathcal{E}$ has fields: `source`, `timestamp`, `signal`, `strength`, `verifiability`

Sources: - **Verified**: assessments, certifications, work samples, portfolio artifacts, manager ratings with rubric - **Behavioral-structured**: structured interview rubric items - **Observed**: performance outcomes, peer feedback (with governance) - **Self-asserted**: resume claims, self-ratings (low reliability by default)

Each source s has a reliability coefficient $r_s \in [0, 1]$.

3.2 Evidence-to-multiplicity update

For each feature f, define an evidence update function U_f producing an increment Δx_f:

$$\Delta x_f \;=\; U_f(e) \;=\; r_{\text{source}(e)} \;\cdot\; \phi_f(e) \;\cdot\; \psi(\Delta t)$$

where: - $\phi_f(e)$ maps evidence strength to a multiplicity increment, - $\psi(\Delta t)$ is a recency factor (e.g., exponential decay).

Boundedness rule For stability and interpretability, impose:

$$0 \le x_f(t) \le X_f^{\max}$$

with $X_f^{\max}$ feature-specific caps.

3.3 Hard skill multiplicity templates

Hard skills are updated via evidence-based templates:

Template HS-1 (work sample / assessment) - If assessment score $q \in [0, 1]$:

$$\Delta x_f = r_s \cdot a_f \cdot q$$

Template HS-2 (experience duration) - If verified experience months m:

$$\Delta x_f = r_s \cdot b_f \cdot \log(1 + m)$$

Template HS-3 (credential / certification) - If credential level $\ell \in \{1, 2, 3\}$:

$$\Delta x_f = r_s \cdot c_f \cdot \ell$$

3.4 Soft trait multiplicity (evidence-backed)

Soft traits require **structured, multi-source evidence**.

Define for a soft trait feature f: - a rubric with items $k = 1..K$, each yielding a score $q_k \in [0, 1]$ - minimum evidence diversity: at least two distinct sources among {interview rubric, peer feedback, work sample observations}

Soft trait update:

$$\Delta x_f = \min\left(X_f^{\max} - x_f,\ r_s \cdot a_f \cdot \mathrm{Avg}(q_1, \ldots, q_K)\right)$$

Self-assertion rule (soft traits) Self-asserted soft traits do not directly increase x_f; instead they update a **prior** for uncertainty (Section 4) until corroborated.

3.5 Role feature construction

Role vectors y_j are built from: - job description parsing (weak evidence) - hiring manager rubric for must-haves (strong evidence) - team context signatures (strong evidence)

Split role requirements: - **Critical** thresholds c_j (must-have) - **Preference** components (nice-to-have)

---

4. Uncertainty representation (PMDM-lite)

4.1 Diagonal uncertainty model

For each candidate i, maintain a diagonal uncertainty vector:

$$\sigma_i \in [0, 1]^F$$

Interpretation: $\sigma_{i,f}$ is uncertainty about $x_{i,f}$ (higher means less certain).

Initialize: - verified evidence $\Rightarrow$ low uncertainty - self-asserted evidence $\Rightarrow$ high uncertainty

Update rule:

$$\sigma_{i,f}(t+1) = (1 - \lambda_f)\, \sigma_{i,f}(t) + \lambda_f\ \exp(-\kappa\, \texttt{evidence\_count}_f)$$

Where λ_f is a feature-specific learning rate.

4.2 Uncertainty-aware intersection

Define an uncertainty-discounted intersection:

$$I_{\mathrm{unc}}(x, y; \sigma) = \sum_{f \in F} w_f \, (1 - \sigma_f) \, \min(x_f, y_f)$$

This causes uncertain claims to contribute less until verified.

4.3 Optional full PMDM

In high-stakes deployments, represent a candidate state as a positive semidefinite matrix ρ over a feature-embedding space, capturing correlations (e.g., clustered skills). This is optional; DRME-X defaults to diagonal uncertainty.

---

5. Stable operator dynamics

5.1 Operator set (bounded semigroup)

Operators act on multiplicity vectors with bounded outputs: - Dilation: $D_\alpha(x) = \alpha x$, with $\alpha \in [\alpha_{\min}, \alpha_{\max}]$ - Mixing: $M_A(x) = Ax$, where $A \geq 0$, sparse, and $\|A\|$ bounded - Projection: $P_{\mathcal{S}}$ keeps a selected feature subset - Context lift: $C_H(x) = x + \gamma \, \mathrm{Agg}_H(x)$, with γ bounded - Normalization/projection: Π enforces nonnegativity, caps, and optional ℓ_1 sparsity

5.2 Evolution law (candidate)

Let evidence stream for candidate i be $e_i(t)$. Then:

$$x_i(t+1) = \Pi\Big(M_{A(t)}(x_i(t)) + U(e_i(t)) + \gamma \, \mathrm{Agg}_H(x_i(t)) \Big)$$

Where $U(e_i(t))$ aggregates $U_f(e)$ over features.

5.3 Evolution law (role)

Roles drift with market and team changes:

$$y_j(t+1) = \Pi\Big(y_j(t) + \Delta y_j^{\text{market}}(t) + \Delta y_j^{\text{manager}}(t) \Big)$$

---

6. Resonance functional (match score)

6.1 Intersection (fit) term

$$I_{ij}(t) = I_{\mathrm{unc}}\big(x_i(t), y_j(t); \sigma_i(t)\big)$$

6.2 Growth alignment

$$G_{ij}(t) = I\big(g_i(t), y_j(t)\big)$$

6.3 Context compatibility

$$K_{ij}(t) = I\big(c_i(t), c_j(t)\big)$$

6.4 Missing-critical penalty

Let c_j be critical thresholds:

$$P_{ij}(t) = \sum_{f \in F} u_f \ \max(0, c_{j,f} - x_{i,f}(t))$$

6.5 Final match score

$$\mathrm{Match}_{ij}(t) = \sigma\Big(\alpha I_{ij}(t) + \beta G_{ij}(t) + \eta K_{ij}(t) - \delta P_{ij}(t) + b_j\Big)$$

Where σ is a squashing function (e.g., logistic) and b_j is a role bias/intercept.

---

7. Explainability payload (required output)

For each recommendation, return: - top contributing intersection features $\arg\max_f \ w_f(1-\sigma_f)\min(x_f, y_f)$ - missing critical requirements and gap sizes - which context factors contributed - uncertainty flags (“this claim is unverified; discount applied”)

---

8. Validation protocol (fast path)

8.1 Offline

- Evaluate ranking quality: NDCG@k, MAP, Recall@k
- Calibrate probabilities: reliability diagrams / Brier score
- Compare against baselines: embedding cosine, LTR, weighted Jaccard

8.2 Online pilot (decision support)

- Measure recruiter time-to-shortlist, interview-to-offer conversion
- Track 90/180-day retention uplift
- Monitor drift and fairness metrics before enabling recursive updates

---

9. Minimal implementation checklist

- [ ] Ontology service (feature registry, prime IDs, versioning)
- [ ] Evidence ingestion + reliability coefficients
- [ ] Sparse vector store for x/y/g/context + uncertainty
- [ ] Hypergraph adjacency + aggregation

- [ ] Resonance scorer + explanation payload
- [ ] Logging, audits, fairness monitoring, drift monitoring